🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

นางสาวกนกพร บุญครอง 673380024-0 sec1

นางสาวชิดชนก ชนะพา 673380033-9 sec1

นางสาวญาทิชา จันทรศรีสุริยวงศ์ 673380034-7 sec1

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0
```

```
end
write memory
```

```
R1>enable
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#hostname R1
R1(config)#interface g0/0
R1(config-if)#noshutdown
                ^
% Invalid input detected at '^' marker.

R1(config-if)#no shutdown
R1(config-if)#interface g0/0.10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0.20
R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console
write memory
Building configuration...
[OK]
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS
```

```
vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
write memory
```

```
Switch>enable
Switch#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#interface g0/1
%Invalid interface type and number
S1(config)#switchport mode trunk
            ^
% Invalid input detected at '^' marker.

S1(config)#interface fa0/1
S1(config-if)#switchport mode trunk

S1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up

S1(config-if)#interface g0/1
%Invalid interface type and number
S1(config)#interface fa2/1
S1(config-if)#switchport mode trunk
S1(config-if)#interface fa0/1
S1(config-if)#switch mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#interface fa 0/2
%Invalid interface type and number
S1(config)#switchport mode access
            ^
% Invalid input detected at '^' marker.

S1(config)#switchport access vlan 20
            ^
% Invalid input detected at '^' marker.

S1(config)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#write memory
Building configuration...
[OK]
```

Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

```
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time=3ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time=1ms TTL=127
Reply from 192.168.20.20: bytes=32 time=1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 3ms, Average = 1ms
```

Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     unassigned      YES unset  up                    up
GigabitEthernet0/0.10  192.168.10.1    YES manual up                    up
GigabitEthernet0/0.20  192.168.20.1    YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

```
S1#show interface trunk
Port        Mode         Encapsulation  Status        Native vlan
Fa2/1       on           802.1q         trunking      1

Port        Vlans allowed on trunk
Fa2/1       1-1005

Port        Vlans allowed and active in management domain
Fa2/1       1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Fa2/1       1,10,20
```

```
S1#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0001.631e.2a01    DYNAMIC     Fa2/1
  10    0001.631e.2a01    DYNAMIC     Fa2/1
  10    0060.7076.c035    DYNAMIC     Fa0/1
  20    0001.631e.2a01    DYNAMIC     Fa2/1
  20    0060.47c3.20a7    DYNAMIC     Fa1/1
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

```
[booboo@Booboos-Macbook-Pro server % python3 main_enhanced.py --host 0.0.0.0 --po]
rt 8888 --storage ../received_files --verbose
[15:03:16] INFO: Starting server with host=0.0.0.0, port=8888, storage=../receiv
ed_files
[15:03:16] INFO: Created storage directory: ../received_files
[15:03:16] INFO: Server listening on 0.0.0.0:8888
[15:03:53] INFO: Connection from ('127.0.0.1', 55031)
[15:03:53] INFO: Receiving text/plain (35 bytes)
[15:03:53] INFO: Saved to ../received_files/received_044e.plain
[15:03:53] INFO: Receiving image/png (108 bytes)
[15:03:53] INFO: Saved to ../received_files/received_4789.png
[15:03:53] ERROR: Failed to write file ../received_files/received_4789.png: [Err
no 32] Broken pipe
[15:03:53] WARNING: Connection error with ('127.0.0.1', 55031): [Errno 32] Broke
n pipe
[15:03:53] INFO: Connection with ('127.0.0.1', 55031) closed
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

```
[booboo@Booboos-Macbook-Pro client % python3 main_enhanced.py ../assets/notes.txt]
 ../assets/sample.png --host 127.0.0.1 --port 8888 --timeout 30 --verbose
[15:03:53] INFO: Starting client with 2 files to 127.0.0.1:8888
[15:03:53] INFO: Connected to 127.0.0.1:8888
[15:03:53] INFO: Sent ../assets/notes.txt (35 bytes) as text/plain
[15:03:53] INFO: Sent ../assets/sample.png (108 bytes) as image/png
[15:03:53] INFO: Connection closed. Sent: 2, Failed: 0
[15:03:53] INFO: All files sent successfully
```

Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

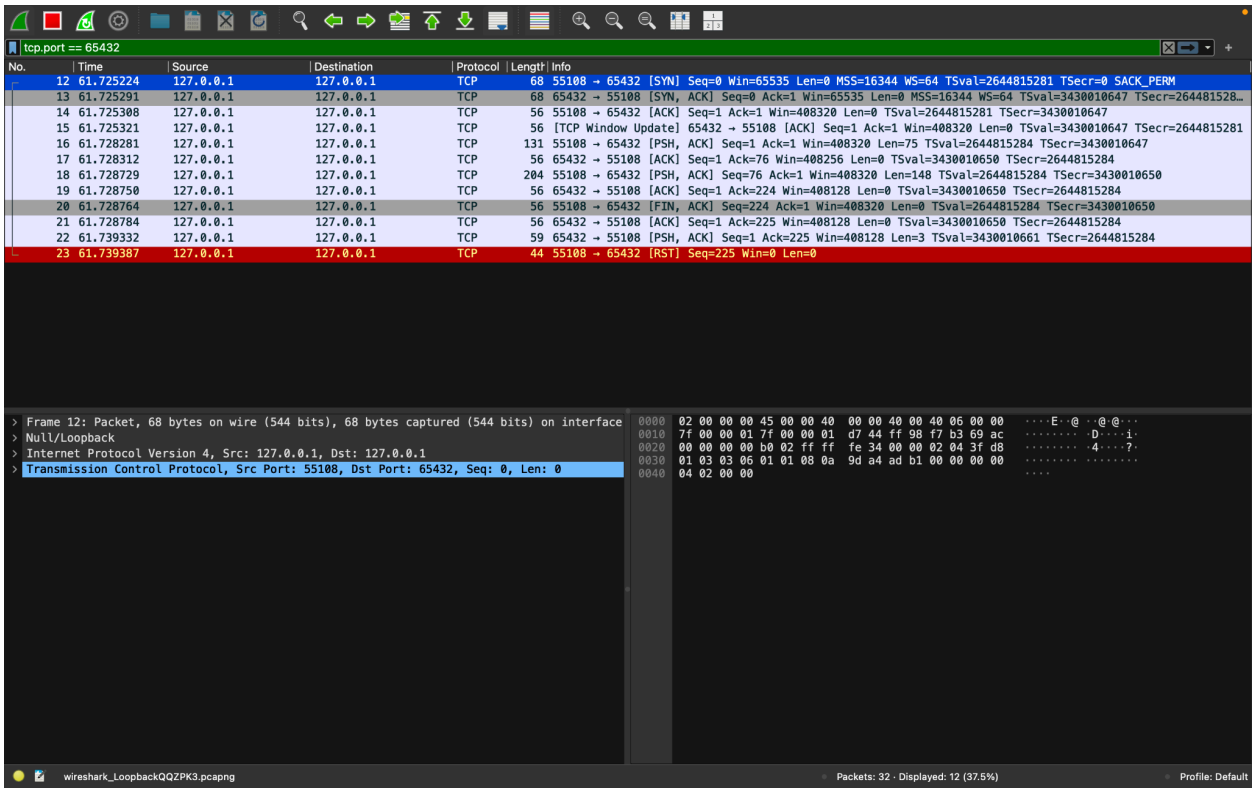
Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.



การทดลองครั้งนี้ถือว่าสำเร็จในด้านการสื่อสารผ่านโปรโตคอล TCP เนื่องจากจากการตรวจสอบแพ็กเก็ตในโปรแกรม Wireshark พบว่ามีการสร้างการเชื่อมต่อระหว่างต้นทางและปลายทางอย่างถูกต้อง โดยปรากฏขั้นตอนการสร้างการเชื่อมต่อแบบ Three-Way Handshake ครบถ้วน ได้แก่ การส่งแพ็กเก็ต SYN จากฝั่งไคลเอนต์

การตอบกลับด้วย SYN-ACK จากฝั่งเซิร์ฟเวอร์ และการยืนยันด้วย ACK จากไคลเอนต์ ซึ่งแสดงให้เห็นว่าทั้งสองฝั่งสามารถติดต่อและตกลงสร้างการเชื่อมต่อ TCP ได้สำเร็จ

Step 3 — Analyze Packets

A — ARP Exchange

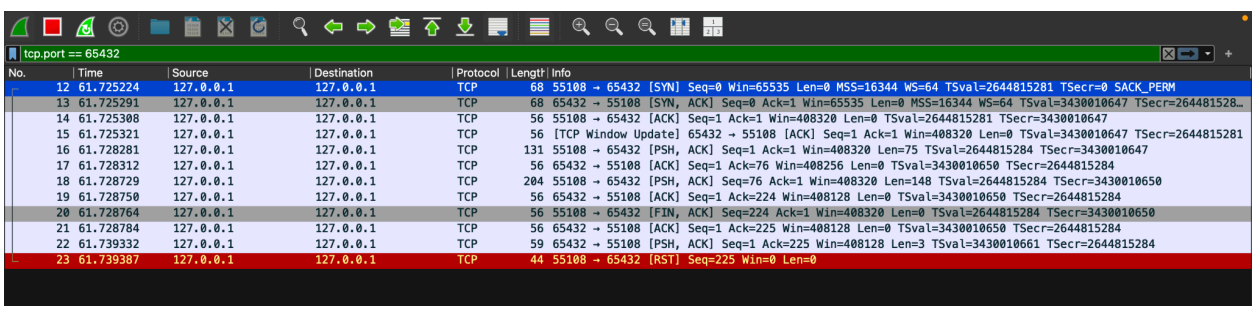
Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.
เนื่องจากการสื่อสารที่เกิดขึ้นเป็นการติดต่อภายในเครื่องเดียวกัน (localhost) โดยใช้ที่อยู่ IP แบบ Loopback คือ 127.0.0.1 ซึ่งไม่ต้องผ่านเครือข่ายจริงหรือการ์ดเครือข่ายภายนอก

B — TCP Three-Way Handshake

Identify:



tcp.port == 65432

No.	Time	Source	Destination	Protocol	Length	Info
12	61.725224	127.0.0.1	127.0.0.1	TCP	68	55108 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2644815281 TSecr=0 SACK_PERM
13	61.725291	127.0.0.1	127.0.0.1	TCP	68	65432 → 55108 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3430010647 TSecr=264481528…
14	61.725308	127.0.0.1	127.0.0.1	TCP	56	55108 → 65432 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2644815281 TSecr=3430010647
15	61.725321	127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 65432 → 55108 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3430010647 TSecr=2644815281
16	61.728281	127.0.0.1	127.0.0.1	TCP	131	55108 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=75 TSval=2644815284 TSecr=3430010647
17	61.728312	127.0.0.1	127.0.0.1	TCP	56	65432 → 55108 [ACK] Seq=1 Ack=76 Win=408256 Len=0 TSval=3430010650 TSecr=2644815284
18	61.728729	127.0.0.1	127.0.0.1	TCP	204	55108 → 65432 [PSH, ACK] Seq=76 Ack=1 Win=408320 Len=148 TSval=2644815284 TSecr=3430010650
19	61.728750	127.0.0.1	127.0.0.1	TCP	56	65432 → 55108 [ACK] Seq=1 Ack=224 Win=408128 Len=0 TSval=3430010650 TSecr=2644815284
20	61.728764	127.0.0.1	127.0.0.1	TCP	56	55108 → 65432 [FIN, ACK] Seq=224 Ack=1 Win=408320 Len=0 TSval=2644815284 TSecr=3430010650
21	61.728784	127.0.0.1	127.0.0.1	TCP	56	65432 → 55108 [ACK] Seq=1 Ack=225 Win=408128 Len=0 TSval=3430010650 TSecr=2644815284
22	61.739332	127.0.0.1	127.0.0.1	TCP	59	65432 → 55108 [PSH, ACK] Seq=1 Ack=225 Win=408128 Len=3 TSval=3430010661 TSecr=2644815284
23	61.739387	127.0.0.1	127.0.0.1	TCP	44	55108 → 65432 [RST] Seq=225 Win=0 Len=0

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.

จากภาพ Wireshark สามารถระบุขั้นตอน TCP Three-Way Handshake ได้ครบทั้ง 3 ขั้นตอน ดังนี้

1. SYN — ไคลเอนต์ (พอร์ต 55108 → 65432) ส่งคำขอเชื่อมต่อ โดยมี Sequence Number = 0

2. SYN-ACK — เซิร์ฟเวอร์ตอบกลับ (65432 → 55108) ยอมรับการเชื่อมต่อ โดยมี Sequence Number = 0 และ Acknowledgment Number = 1 (ยืนยัน SYN ของไคลเอนต์)

3. ACK — ไคลเอนต์ยืนยันขั้นสุดท้าย (55108 → 65432) โดยมี Sequence Number = 1 และ Acknowledgment Number = 1

สรุป: การจับมือสามทางสำเร็จ เนื่องจากค่า Sequence และ Acknowledgment Numbers เพิ่มขึ้นตามหลักของ TCP ทำให้ทั้งสองฝั่งพร้อมส่งข้อมูลต่อไป

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.

```
{"mime_type": "text/plain", "size": 35}
Test content for final validation.
{"mime_type": "image/png", "size": 108}
.PNG
.
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
OK
```

D — VLAN Tags

If capturing on trunk:

Look for:

```
None
802.1Q Virtual LAN
```

Note VLAN ID.

ในการทดลองส่วน D — VLAN Tags หากเป็นการดักจับข้อมูลบนพอร์ตแบบ Trunk ตามปกติควรจะพบเฟรมที่มีแท็ก IEEE 802.1Q Virtual LAN ซึ่งใช้ระบุหมายเลข VLAN (VLAN ID) ภายในเครือข่าย อย่างไรก็ตาม จากการทดลองนี้ไม่พบข้อมูล 802.1Q หรือ VLAN Tag ใด ๆ ในแพ็กเก็ตที่จับได้

สาเหตุเนื่องจากการทดลองทำบนเครื่องเดียวกันผ่าน Loopback Interface ไม่ได้ผ่านสวิตช์หรือพอร์ต Trunk จริง เฟรมจึงไม่ถูกติดแท็ก VLAN และไม่มี VLAN ID ปรากฏในข้อมูลที่จับได้

สรุปได้ว่า ไม่พบ 802.1Q Virtual LAN Tag และไม่สามารถระบุ VLAN ID ได้ เนื่องจากไม่ได้ดักจับทราฟฟิกจากเครือข่ายที่มีการใช้งาน VLAN จริง แต่เป็นการสื่อสารภายในเครื่องเท่านั้น

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

เมื่อทำการ shutdown subinterface จะทำให้เส้นทางเครือข่ายของ VLAN หรือเครือข่ายย่อยนั้นใช้งานไม่ได้ ส่งผลให้การสื่อสารระหว่างเครือข่ายล้มเหลวทันที

ผลที่สังเกตได้คือ Ping จะไม่สำเร็จ (Request timed out) เนื่องจากแพ็กเก็ต ICMP ไม่สามารถถูกส่งไปถึงปลายทางหรือไม่มีเส้นทางกลับ ขณะเดียวกันการเชื่อมต่อแบบ TCP จะเกิด TCP timeout เพราะไม่สามารถสร้างการเชื่อมต่อหรือรับการตอบกลับได้ภายในเวลาที่กำหนด

สรุป: การปิด subinterface ทำให้การ routing สำหรับเครือข่ายนั้นหยุดทำงาน ส่งผลให้ทั้ง Ping และการเชื่อมต่อ TCP ล้มเหลวเนื่องจากไม่มีเส้นทางสื่อสารไปยังปลายทาง

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

เมื่อไฟร์วอลล์ทำการ บล็อกพอร์ตปลายทาง แพ็กเก็ต TCP ที่ส่งไปเพื่อเริ่มการเชื่อมต่อจะไม่สามารถผ่านไปถึงเซิร์ฟเวอร์ได้ หรือถูกทิ้งโดยไม่มีการตอบกลับ ส่งผลให้ไคลเอนต์ไม่ได้รับแพ็กเก็ต SYN-ACK กลับมา

สิ่งที่สังเกตได้คือจะเกิด SYN retransmissions หรือการส่ง SYN ซ้ำหลายครั้ง เนื่องจาก TCP พยายามเริ่มการเชื่อมต่อใหม่เมื่อไม่ได้รับการตอบสนอง ภายในระยะเวลาหนึ่ง หากยังไม่มีการตอบกลับ การเชื่อมต่อจะล้มเหลวและเกิด timeout ในที่สุด

สรุป: การบล็อกพอร์ตโดยไฟร์วอลล์ทำให้ TCP ไม่สามารถสร้างการเชื่อมต่อได้ จึงเกิดการส่ง SYN ซ้ำ ๆ เพื่อพยายามติดต่อปลายทาง แต่ไม่สำเร็จ

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

เมื่อกำหนด Default Gateway ผิด เครื่องจะพยายามส่งข้อมูลออกไปยัง Gateway ที่ไม่มีอยู่จริง หรือไม่สามารถเข้าถึงได้ ทำให้ไม่สามารถส่งแพ็กเก็ตออกนอกเครือข่ายได้

สิ่งที่สังเกตได้คือสถานะ ARP incomplete เนื่องจากเครื่องพยายามค้นหา MAC Address ของ Gateway นั้นด้วย ARP แต่ไม่ได้รับการตอบกลับ จึงไม่สามารถทำ MAC resolution ได้ และไม่สามารถส่งข้อมูลต่อไปได้

สรุป: Gateway ที่ตั้งผิดทำให้เครื่องหา MAC Address ของปลายทางไม่พบ เกิด ARP incomplete และการสื่อสารไปยังเครือข่ายอื่นล้มเหลว

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

เมื่อมีการจำลอง Packet Loss (การสูญหายของแพ็กเก็ต) โดยการดรอปแพ็กเก็ตบางส่วนระหว่างทาง TCP จะตรวจพบว่าข้อมูลไปไม่ถึงปลายทางเนื่องจากไม่ได้รับ ACK ตามที่คาดไว้

สิ่งที่สังเกตได้คือจะเกิด TCP retransmissions หรือการส่งแพ็กเก็ตเดิมซ้ำ เพื่อให้ข้อมูลถึงปลายทางครบถ้วน นอกจากนี้ TCP จะมีการ ปรับขนาดหน้าต่างรับส่งข้อมูล (Window adjustment) โดยลดขนาดหน้าต่างลงเพื่อลดอัตราการส่งข้อมูลและหลีกเลี่ยงการสูญหายเพิ่มเติม เมื่อเครือข่ายกลับมาปกติ ขนาดหน้าต่างอาจค่อย ๆ เพิ่มขึ้นอีกครั้ง

สรุป: การสูญหายของแพ็กเก็ตทำให้ TCP ต้องส่งข้อมูลซ้ำและปรับความเร็วการส่งผ่านการเปลี่ยนขนาดหน้าต่าง เพื่อรักษาความถูกต้องและเสถียรภาพของการสื่อสาร

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?
   TCP ส่งข้อมูลซ้ำเมื่อข้อมูลสูญหาย เพื่อให้การสื่อสารมีความน่าเชื่อถือ ข้อมูลต้องไปถึงครบและถูกต้อง แม้เครือข่ายจะมีความผิดพลาด

2. How does VLAN tagging affect broadcast domains?
   การติดแท็ก VLAN ทำให้เครือข่ายถูกแบ่งเป็นหลาย broadcast domain อุปกรณ์คนละ VLAN จะไม่เห็น broadcast ของกันและกัน ต้องผ่านเราเตอร์จึงจะสื่อสารข้าม VLAN ได้

3. What role does MIME play compared to TCP headers?
   MIME ทำหน้าที่บอก “ความหมายของข้อมูล” ส่วน TCP header ทำหน้าที่ดูแลการขนส่งให้ข้อมูลถึงปลายทางอย่างถูกต้อง

4. Where does routing decision occur?
   การตัดสินใจเส้นทางเกิดขึ้นที่เราเตอร์ โดยดูจาก IP ปลายทางและตารางเส้นทาง

5. Why does ARP precede TCP communication?
   ต้องใช้ ARP หา MAC address ก่อน ถึงจะส่งแพ็กเก็ต TCP ผ่านเครือข่ายท้องถิ่นได้

📊 Reflection Exercise

Explain:

> How does application meaning survive multiple encapsulation layers?

ความหมายของข้อมูลระดับแอปพลิเคชัน (เช่น ข้อความ รูปภาพ คำสั่ง หรือข้อมูลธุรกรรม) ยังคง “ไม่เปลี่ยนแปลง” แม้ว่าจะถูกห่อหุ้ม (encapsulation) ผ่านหลายชั้นของโมเดลเครือข่าย (เช่น TCP/IP หรือ OSI) เพราะเหตุผลสำคัญดังนี้:

1) แต่ละชั้นเพิ่ม “ส่วนควบคุม” โดยไม่แก้ไขข้อมูลเดิม

เมื่อข้อมูลถูกส่งลงแต่ละชั้น จะมีการเพิ่ม Header (และบางครั้ง Trailer) เข้าไปด้านหน้า/ด้านหลัง แต่ Payload เดิมยังคงเหมือนเดิมทุกประการ

เปรียบเหมือนใส่จดหมายลงซองหลายชั้น

- ตัวจดหมาย = ความหมายของแอปพลิเคชัน
- ซองแต่ละชั้น = Header ของแต่ละเลเยอร์

2) แต่ละชั้นทำหน้าที่เฉพาะของตนเอง

แต่ละเลเยอร์ดูแลเรื่องที่ต่างกัน เช่น:

- Application: ความหมายของข้อมูล
- Transport: การแบ่งข้อมูลและความน่าเชื่อถือ
- Network: การกำหนดเส้นทาง
- Data Link: การส่งในเครือข่ายท้องถิ่น
- Physical: การแปลงเป็นสัญญาณไฟฟ้าหรือคลื่น

ไม่มีชั้นใด “ตีความ” ความหมายของข้อมูลแอปพลิเคชัน ยกเว้นชั้น Application ปลายทาง

3) การถอด encapsulation (Decapsulation) ที่ปลายทาง

เมื่อข้อมูลถึงผู้รับ จะถูกแกะออกทีละชั้นในลำดับย้อนกลับ

Physical → Data Link → Network → Transport → Application

ท้ายที่สุด Payload เดิมจะถูกส่งให้โปรแกรมปลายทางเหมือนตอนส่งออกมา

4) มีการตรวจสอบความถูกต้องระหว่างทาง

หลายชั้นมีระบบตรวจสอบข้อผิดพลาด เช่น checksum หรือ ACK เพื่อให้มั่นใจว่าข้อมูลไม่เสียหาย หากผิดพลาดอาจมีการส่งใหม่

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer

🧬 Instructor Notes — What This Lab Really Teaches

This is not just networking.

It demonstrates:

- Protocol layering philosophy
- Reliability engineering
- Metadata semantics

- Distributed system fundamentals

The student watches a file become:

intention → bytes → frames → signals → bytes → meaning

That transformation is the essence of computer networks.